TED UNIVERSITY

Department of Computer Engineering

MoonAI
Neuroevolution Prey-Predator Simulation

Final Report

Team:

Caner Aras Emir Irkılata Oğuzhan Özkaya

Supervisor:

Dr. Ayşenur Birtürk

Jury Members:

Asst. Prof. Deniz Cantürk Asst. Prof. Mehmet Evren Coşkun

CMPE 492 — Senior Project II

Spring 2026

Table of Contents

1. Introduction and Project Context

This final report presents the completed architecture, implementation approach, validation summary, and engineering assessment of the senior project titled **Hybrid Intelligence for Deep Semantic Analysis of UAV Thermal Imagery in Closed Networks**. The project was developed as a decision-support system for security-oriented scenarios in which external connectivity is either prohibited or operationally unsafe.

The central engineering problem addressed by the project is the gap between *detection* and *understanding*. Existing surveillance pipelines can often locate objects such as people or vehicles, but they frequently fail to explain their likely role, status, or threat relevance. For thermal imagery this gap is even larger because the input is noisy, low in texture, and fundamentally different from standard RGB imagery.

The project therefore adopted a hybrid architecture: a fast detector first scans the full thermal scene and identifies candidate objects, after which a vision-language reasoner performs deeper analysis on cropped regions of interest. In practical terms, the system is designed to move from outputs such as "vehicle detected" toward outputs such as "light pickup-type vehicle with an active front thermal signature" or "human carrying a long, rifle-like object."

1.1. Project Motivation

Thermal imagery is highly relevant for night operations, wide-area reconnaissance, and degraded visual environments. However, most modern multimodal models are trained mainly on RGB imagery and therefore inherit a strong bias toward color, texture, and ordinary lighting conditions. This creates a mismatch between the strengths of general-purpose visual models and the actual needs of thermal UAV analysis.

At the same time, security-sensitive deployments often cannot rely on cloud services. A closed-network architecture is desirable for data sovereignty, reduced latency, and operational robustness. These two realities together motivated a design that is both **thermal-aware** and **offline-capable**.

1.2. Project Objectives

The final project was organized around the following objectives:

- detect objects of interest in thermal UAV imagery with a model specialized for fast scene scanning,
- perform deeper semantic interpretation on detected targets rather than relying only on coarse labels,
- operate fully inside a local or air-gapped network without dependence on external APIs or cloud inference,
- produce structured outputs suitable for dashboards, logs, and post-mission review,
- preserve a human-in-the-loop decision model rather than replacing the operator.

1.3. Project Context

The work aligns with the broader goal of building secure and locally deployable artificial intelligence systems for defense and critical-infrastructure monitoring. Within this context, the project treated the software as a decision-support mechanism, not as an autonomous action system. The operator remains responsible for interpreting alerts and making any mission-critical decision.

The project also fits the educational purpose of the senior design sequence: it combines requirements engineering, system design, literature review, model selection, deployment planning, testing, and ethical assessment within a single integrated capstone effort.

1.4. Scope of This Report

This report documents:

- the final problem framing and literature background,
- the requirements and constraints that shaped the system,
- the implemented two-stage architecture,
- the tools and technologies studied and applied,
- the test results and their engineering interpretation,
- the global, economic, environmental, societal, and ethical implications of the work.

2. Background and Related Work

The project sits at the intersection of thermal computer vision, UAV surveillance, and vision-language reasoning. The literature reviewed during the project showed that these three areas have each progressed rapidly, but they are not yet naturally aligned for secure, offline, semantic analysis of thermal imagery.

2.1. Why Thermal UAV Imagery Is Difficult

Thermal imagery differs fundamentally from ordinary RGB imagery. It contains heat patterns instead of color, often has reduced texture, and is especially vulnerable to ambiguity when small or distant objects are observed from aerial platforms. These properties affect both detection quality and downstream reasoning quality.

The main challenges are:

- low contrast and limited fine detail,
- absence of color cues and weak texture information,
- noise introduced by the sensor and environment,
- small target size in wide-area aerial scenes,
- domain mismatch between thermal imagery and RGB-pretrained multimodal models.

These challenges explain why a naive “send the whole thermal frame to a VLM” strategy is

rarely sufficient.

2.2. Detection-Focused Literature

Thermal object detection itself is a relatively mature subproblem compared with thermal semantic reasoning. The YOLO family remains one of the most widely adopted choices for real-time detection because of its balance between speed and accuracy [1]. Public datasets such as FLIR ADAS, KAIST Multispectral, HIT-UAV, DroneVehicle, and LLVIP [9, 10, 11, 12, 13] provide the data basis for adapting detectors to thermal or cross-modal imagery.

Among these resources, HIT-UAV is particularly relevant because it targets UAV-based infrared observation and therefore resembles the project scenario more closely than ordinary ground-camera datasets [11]. DroneVehicle is also important because it shows that aerial cross-modal detection can be treated as an engineering problem with dedicated datasets and model design choices [12].

The literature review therefore suggested a clear conclusion: *finding targets in thermal imagery is feasible with specialized detectors*. The harder problem is what happens after the target is found.

2.3. Vision-Language Reasoning Models

Open multimodal models such as Qwen2-VL, LLaVA-NeXT, and InternVL demonstrate strong reasoning ability on general image understanding tasks [2, 3, 4]. However, their strongest published results come mainly from datasets closer to RGB photography, document understanding, or general web imagery than to aerial thermal reconnaissance.

For this reason, the project did not treat the VLM as a replacement for detection. Instead, it treated the VLM as a second-stage reasoning engine. This design directly addresses three thermal failure modes that were repeatedly identified during the background study:

- **RGB bias:** general multimodal models are optimized for ordinary visual concepts rather than heat signatures,
- **small-object blindness:** a full-frame thermal scene contains many distant and low-detail targets that are easy for a detector to localize but difficult for a VLM to find,
- **semantic uncertainty:** even when a VLM notices a hot region, it may not reliably distinguish whether it represents a vehicle, a person, or irrelevant thermal clutter.

This is why the final architecture combined a detection expert and a reasoning expert rather than pursuing a single-model design.

2.4. Use of Library and Internet Resources

Background research for the project relied on a combination of academic and technical resources:

- dataset papers and official dataset pages for FLIR ADAS, KAIST, HIT-UAV, DroneVe-

hicle, and LLVIP,
- official model papers and project pages for YOLOv8, Qwen2-VL, LLaVA-NeXT, and InternVL,
- official documentation for PyTorch, OpenCV, and NVIDIA Jetson platforms,
- professional and ethical references from ACM and IEEE.

These resources served different purposes. Academic papers were used to understand the state of the art and comparable designs. Official documentation was used to understand component capabilities, deployment constraints, and integration requirements. Ethical codes and engineering standards were used to frame the system as a decision-support tool rather than an autonomous authority.

2.5. Key Takeaway from the Review

The literature review did not suggest that one new model was needed. Instead, it suggested that the most practical and defensible solution was **architectural**: use a detector to answer "where is the target?" and a vision-language model to answer "what does the target likely mean?" This directly motivated the final two-stage "Gözcü + Analist" pipeline.

3. Requirements, Constraints, and Final Scope

The final system was shaped by a combination of functional expectations, operational constraints, and practical scope limits, including object detection, deep semantic reasoning, offline execution, structured output generation, and operator-facing presentation.

3.1. Functional Requirements

- **FR-1:** The system shall ingest thermal imagery from UAV or UAV-like local video sources.
- **FR-2:** The system shall detect multiple objects of interest in a frame, including at least people, vehicles, animals, weapons, or ammunition-related classes where supported by training data.
- **FR-3:** The system shall crop regions of interest from detector outputs and pass them to a second-stage reasoning component.
- **FR-4:** The system shall generate semantic analysis richer than plain labels, such as likely object type, possible status, or qualitative threat relevance.
- **FR-5:** The system shall produce structured machine-readable output in JSON form for logging and interface consumption.
- **FR-6:** The system shall operate without dependence on external web APIs or cloud inference during runtime.
- **FR-7:** The system shall support local presentation of alerts, detections, or analytical summaries for the operator.

3.2. Non-Functional Requirements

- **Reliability:** The pipeline should behave consistently on representative thermal scenes and should degrade gracefully when no object is found.
- **Security:** Data acquisition, processing, and storage must remain inside a local or air-gapped environment with zero intentional external transfer.
- **Performance:** The design should remain compatible with near-real-time operation and edge deployment goals, even if final benchmarking depends on hardware availability.
- **Usability:** The operator output should be more actionable than raw detections and should reduce interpretation burden rather than increase it.
- **Maintainability:** The architecture should remain modular so that the detector, reasoner, prompts, or logging layer can be updated independently.

3.3. Project Constraints

Several constraints influenced the final design choices:

- **Closed-network constraint:** the system could not rely on remote inference or hosted model APIs.
- **Thermal-domain constraint:** training data and model behavior are limited by the intrinsic ambiguity of thermal imagery.
- **Hardware constraint:** the target vision involved deployment on resource- constrained edge hardware such as the NVIDIA Jetson family.
- **Open-source constraint:** the project aimed to use publicly available models, frameworks, and datasets wherever possible.
- **Capstone scope constraint:** the project needed to stay feasible within a senior project schedule rather than expand into full-scale field qualification.

3.4. Final Scope Boundaries

The final scope emphasized the following boundaries are important:

- The system aims for *contextual understanding*, but not guaranteed forensic- grade identification of exact commercial vehicle make and model.
- The system is a *decision-support* tool and does not automate engagement or tactical response.

These scope boundaries are consistent with both the project's educational setting and the real technical difficulty of thermal semantic reasoning.

4. Final System Architecture and Design

The final system architecture is a two-stage hybrid pipeline designed to combine the speed of a specialized thermal detector with the reasoning depth of an open vision-language model. This design was chosen because a single general-purpose model could not simultaneously

deliver reliable thermal target localization, low-latency scanning, and semantically useful interpretation.

4.1. Architecture Overview

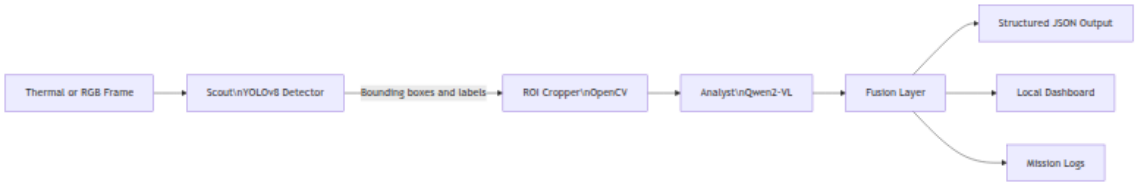

Figure 1: High-level system pipeline used in the final prototype.

The architecture begins with a thermal frame, applies full-frame detection, isolates each candidate region, performs target-focused reasoning, and then fuses the results into structured output. This decomposition reflects the key engineering principle of the project: *detect first, interpret second*.

4.2. Two-Stage "Gözcü + Analist" Pipeline

The two main stages are:

Table 1: Primary roles of the two-stage pipeline

Stage	Component	Responsibility
Stage 1	Gözcü (Scout)	Scan the full scene quickly, detect objects of interest, and return bounding boxes, class labels, and confidence values.
Stage 2	Analist (Analyst)	Receive cropped target regions and infer deeper semantic meaning such as likely object type, possible engine activity, carried-object clues, or qualitative threat relevance.

This role separation is important. The detector answers "where is the object?" and "what is the coarse class?" The reasoner answers "what does this object likely mean in context?"

4.3. Subsystem Decomposition

The final architecture can be understood as five subsystems.

- **Data acquisition:** Accept local thermal or RGB streams, static images, or frame sequences without requiring external network services.
- **Detection:** Run a thermal-adapted YOLOv8 model on the full frame and generate detections suitable for downstream processing.
- **ROI processing:** Crop and optionally enhance the detected object region so the second stage focuses on the target instead of the whole scene.

- **Semantic reasoning:** Use Qwen2-VL with a prompt-driven analysis workflow to produce richer descriptions and infer status-oriented details.
- **Fusion and presentation:** Merge detector and reasoner outputs into JSON, logs, and local interface elements for operator review.

4.4. Low-Level Deployment Packages

Beyond the logical architecture, the implementation protocol was organized into deployment-oriented packages so that hardware-specific concerns could be isolated from application-level reasoning logic.

- **Containerization package:** Encapsulates the runtime environment, model dependencies, and platform-specific libraries inside a reproducible Docker image for Jetson-class deployment.
- **Optimization package:** Applies runtime acceleration techniques such as TensorRT or Torch-TensorRT compilation, layer fusion where applicable, and reduced- precision execution paths for edge hardware.
- **Inference pipeline package:** Coordinates frame ingestion, detector execution, region cropping, reasoning invocation, and result fusion under one bounded runtime flow.
- **Logging and presentation package:** Stores structured JSON outputs locally and exposes operator-facing summaries, alerts, or dashboard elements without external services.

This decomposition is valuable because it separates *what* the system does from *how* it is deployed. The detector and reasoner remain conceptual building blocks, but the deployment packages define how those blocks are actually started, optimized, and operated on edge devices.

4.5. Secure Deployment in a Closed Network

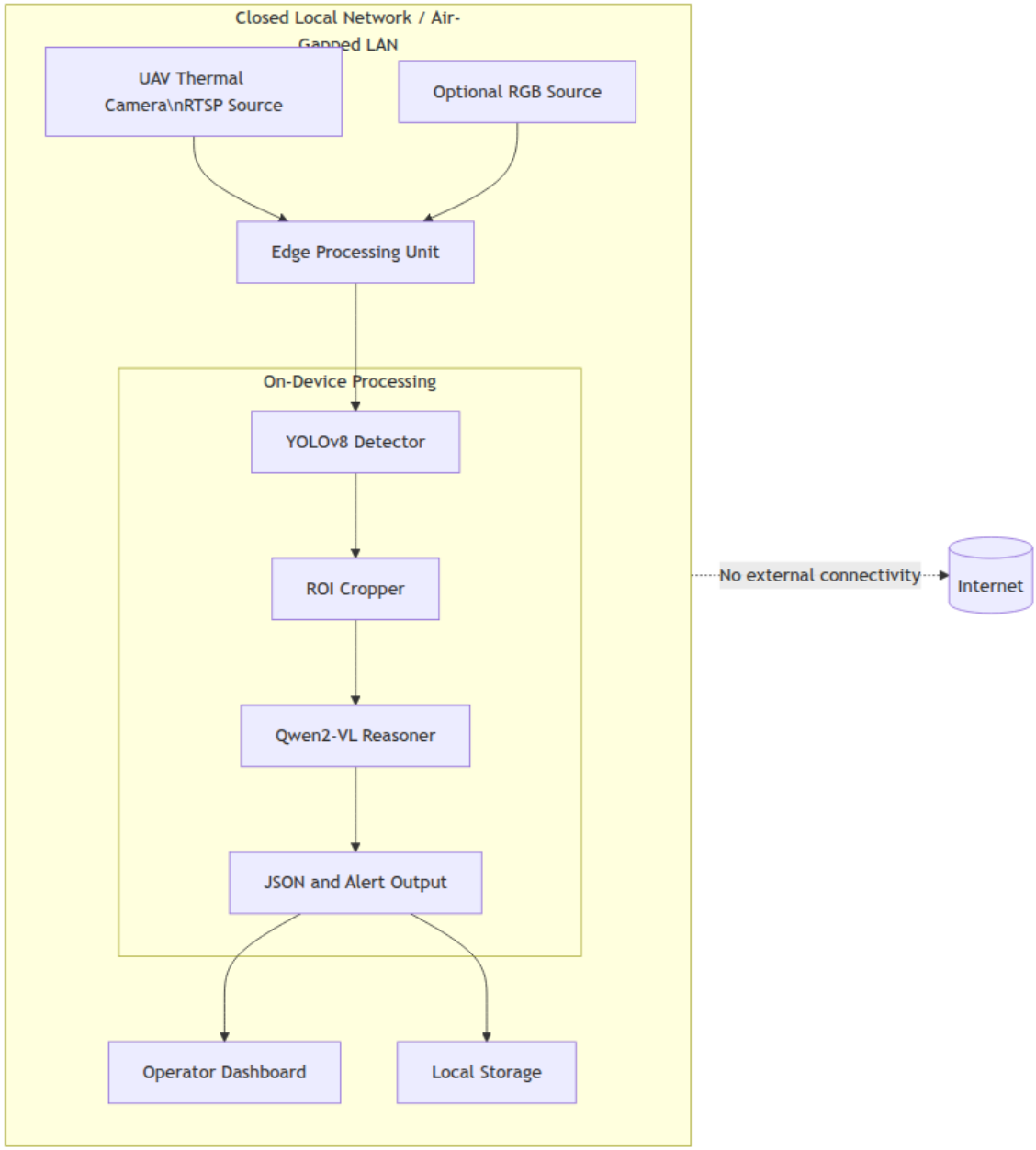

Figure 2: Closed-network deployment concept with local processing and no external connectivity.

The deployment concept keeps image ingestion, model inference, result fusion, and storage within the same local environment. This provides three advantages:

- sensitive data does not leave the operational network,
- latency is reduced because inference is local,
- the system remains usable in environments where internet access is unavailable or intentionally disabled.

At low level, this also affects device integration. Video ingress must be available from a local camera or local network stream, model binaries must reside on the target device or an

authorized internal store, and local visualization inside a containerized runtime may require explicit device and display mapping between host and container.

4.6. Control Flow

The internal software flow of one processed frame is summarized below.

1. Acquire a frame from a local thermal source.
2. Run the detector and obtain a list of candidate objects.
3. For each detection, crop the relevant region of interest.
4. Send the cropped region and an analysis prompt to the VLM.
5. Receive qualitative reasoning output.
6. Merge detector data and reasoner data into a structured record.
7. Display and/or store the final result locally.

4.7. Interface Contracts

The system is easier to maintain when each stage exposes a narrow and typed interface. The final implementation protocol therefore treated component boundaries as explicit contracts, even when the internal implementations evolved.

- **`ingest_frame(source)`**: Returns a raw frame buffer from a local stream, file, or camera source.
- **`detect_objects(frame)`**: Accepts a preprocessed frame and returns bounding boxes, class labels, and confidence values.
- **`crop_regions(frame, detections)`**: Converts detector outputs into isolated regions of interest suitable for second-stage analysis.
- **`analyze_region(region, prompt)`**: Produces a structured semantic description for one target crop.
- **`fuse_results(detections, analyses)`**: Merges low-level detector data and higher-level reasoning output into a final JSON record.

The exact tensor dimensions are implementation-dependent, but the important engineering rule is that each boundary must preserve shape, data type, and memory ownership clearly enough to avoid runtime mismatch between preprocessing, inference, and fusion layers.

4.8. Structured Output Format

The final output is intentionally machine-readable so that it can be consumed by dashboards, logs, or future command-and-control integrations.

```
{
  "objects": [
    {
      "type": "armed_person",
      "bounding_box": [x1, y1, x2, y2],
```

```
      "confidence": 0.92,
      "analysis": {
        "weapon_type": "rifle",
        "risk_level": "high",
        "explanation": "Thermal silhouette suggests a long
           object consistent with a rifle."
      }
    },
    {
      "type": "vehicle",
      "bounding_box": [x3, y3, x4, y4],
      "confidence": 0.85,
      "analysis": {
        "vehicle_category": "pickup_truck",
        "engine_status": "hot",
        "explanation": "Front thermal concentration suggests
           recent engine activity."
      }
    }
  ]
}
```

Listing 1: Illustrative structured output produced by the fusion layer

4.9. Design Rationale

The final design was selected because it balances feasibility and ambition.

- A standalone VLM is too weak at thermal scene scanning.
- A standalone detector is too shallow for contextual interpretation.
- A two-stage pipeline is modular, explainable, and easier to improve incrementally.
- Local JSON output supports both immediate use and later auditability.

The result is not merely a collection of models, but a deliberately engineered workflow that turns thermal pixels into structured, reviewable intelligence.

5. Implementation, Tools, and Technologies

The project was implemented by combining open-source vision, multimodal reasoning, and edge-deployment techniques. The main technical challenge was not inventing a new model from scratch, but selecting and integrating the right tools for the problem. As the low-level design matured, the implementation focus expanded from model selection alone to reproducible packaging, hardware-aware optimization, and explicit runtime contracts for Jetson-class de-

vices.

5.1. Main Technologies Used

Table 2: Core tools and technologies used in the project

Layer	Technology	Role in the project
Detection	YOLOv8	Fast object detection on thermal imagery with strong practical tooling and broad community support [1].
Reasoning	Qwen2-VL 7B	Second-stage vision-language reasoning model used to interpret cropped thermal targets in more detail [2].
Adaptation	LoRA	Parameter-efficient fine-tuning strategy for adapting the reasoning model without retraining all weights [5].
Containerization	Docker / buildx	Reproducible deployment environment and cross- architecture image generation from workstation-class development hosts to ARM-based edge targets [6].
Optimization runtime	TensorRT / Torch-TensorRT	Hardware-aware inference acceleration through engine compilation, mixed precision, and memory-conscious execution strategies [7, 8].
Image processing	OpenCV	Region cropping, pre-processing, and data preparation utilities [16].
Deep learning runtime	PyTorch	Model development, loading, and experimentation framework [17].
Deployment target	NVIDIA Jetson family	Edge-oriented hardware target for local, low-latency inference in a closed network [18].
Data interchange	JSON	Structured output format for alerts, logs, and downstream consumption.

5.2. Detector Layer

YOLOv8 was selected for the detector layer because the problem requires rapid scanning of an entire scene before any deeper reasoning becomes possible. The model family is practical for prototype work because it has mature training and inference tooling, strong existing documentation, and published success on related thermal or aerial detection tasks.

The detector is responsible for:

- locating candidate targets in the full frame,
- producing bounding boxes and coarse class labels,
- acting as the gatekeeper for the more expensive reasoning stage.

This means the detector is not merely a preprocessing step; it is the part of the system that protects runtime performance and gives the VLM a focused visual problem.

From a deployment perspective, the detector is also the component that benefits most directly from TensorRT-style optimization because it is invoked continuously on the full frame and therefore dominates the first stage of the runtime budget.

5.3. Reasoning Layer

Qwen2-VL was selected as the primary reasoning model because it provides strong open multimodal performance, supports high-resolution inputs, and is well suited for prompt-based description and inference tasks [2]. In this project, the role of the model is not to rediscover the object location but to examine the isolated target and produce a more useful interpretation.

Typical reasoning goals include:

- narrowing "vehicle" to a more specific category such as pickup-type or light utility vehicle,
- inferring whether a strong localized thermal signature suggests recent engine activity,
- interpreting whether a human figure appears to be carrying a rifle-like object,
- converting low-level visual evidence into natural-language explanation.

Alternative open models such as LLaVA-NeXT and InternVL were also studied as comparison points because they are strong open multimodal baselines with useful reasoning capability [3, 4]. However, Qwen2-VL remained the most aligned with the project's need for fine-grained visual reasoning under a relatively efficient open-source setup.

5.4. LoRA Fine-Tuning and Domain Adaptation

One of the most important technical lessons of the project was that thermal adaptation is a separate engineering problem from general multimodal reasoning. A VLM trained mostly on ordinary web images does not automatically understand that thermal brightness represents heat, not visible texture or color.

LoRA was therefore important as a strategy because it enables targeted adaptation of a large model with lower computational cost than full retraining [5]. Even when the final prototype is evaluated primarily at system level, the project's design logic remained clear: the analyst model benefits from thermal-domain adaptation, especially when asked to produce object-specific explanations rather than generic captions. This emphasis on adaptation is also consistent with broader thermal-domain transfer work, where modality mismatch is one of the core causes of performance loss [15].

5.5. Containerized Deployment Workflow

To make the implementation portable across workstation and Jetson environments, the low-level deployment workflow was organized around containerization. This has several practical advantages:

- model dependencies are frozen in a reproducible runtime image,
- Jetson-specific library versions can be aligned with the target JetPack or L4T stack,
- the same application pipeline can be developed on x86 hosts and prepared for ARM deployment through cross-architecture image builds.

The use of `docker buildx` is particularly valuable in this context because it bridges the gap between high-resource development machines and ARM-based edge targets. Instead of compiling every dependency directly on the embedded device, the build process can produce a more controlled and repeatable deployment artifact in advance.

5.6. TensorRT Optimization Strategy

The implementation protocol treated acceleration as a structured workflow rather than a single toggle. In edge deployment, optimization typically proceeds through three related steps:

1. graph-level simplification and layer fusion where supported,
2. reduced-precision execution such as FP16 for lower latency and smaller memory use,
3. engine-level runtime specialization for the target NVIDIA hardware.

In this project, such optimizations are most naturally aligned with the detector stage and other repeated tensor operations. For the reasoning stage, the analogous concerns are quantization, reduced memory footprint, staged loading, and careful balancing between semantic depth and runtime cost.

5.7. Supporting Software Components

OpenCV and PyTorch were used as the main supporting frameworks. OpenCV fits the bridge layer of the architecture because cropping and preparing regions of interest is a direct image-processing task. PyTorch fits the model layer because it is the de facto standard framework for open detector and multimodal model experimentation.

JSON was used as the common output format because it offers a simple contract between vision, reasoning, logging, and interface components. A structured format was necessary to avoid producing outputs that were only human-readable but difficult to archive or integrate.

At implementation level, these supporting components also define memory boundaries. Raw frames, normalized detector tensors, cropped ROIs, analysis objects, and final JSON records must move between packages without ambiguity over format or ownership.

5.8. Edge Optimization

The project targeted edge-oriented operation rather than cloud dependence. This led to an interest in quantization and model-size reduction strategies, especially for the reasoning model. In principle, INT4-style quantization is attractive because it reduces memory pressure and improves the feasibility of local deployment on Jetson-class hardware. This does not eliminate the challenge of multimodal inference at the edge, but it makes the deployment goal far more realistic.

Recent Jetson benchmarking work also shows that throughput, memory headroom, and energy budget can vary substantially across embedded hardware classes, which is why edge AI design cannot treat deployment as an afterthought [14]. For this reason, the project considered

memory pressure and runtime continuity as first-class concerns alongside model accuracy.

5.9. Memory Management and Runtime Discipline

Intermediate buffers can become as important as model weights in a resource-constrained deployment. The implementation protocol therefore benefits from:

- explicit staging between frame ingestion, detection, cropping, and reasoning,
- avoiding unnecessary duplication of image tensors,
- conditional execution of the reasoning stage only when the detector produces targets,
- reduced-precision and engine-compiled execution where it provides a stable gain.

5.10. Why These Technologies Matter

Each tool solved a specific problem:

- YOLOv8 solved rapid thermal localization.
- Qwen2-VL solved semantic interpretation beyond labels.
- LoRA solved practical domain adaptation.
- OpenCV solved region isolation and pre-processing.
- PyTorch solved model experimentation and integration.
- Jetson-oriented optimization solved the closed-network edge-deployment goal.

6. Test Plan Summary and Results

6.1. Reconstructed Validation Scope

The retained validation cases covered five areas:

- input and detection,
- target isolation and semantic reasoning,
- output generation and local operation,
- deployment-oriented runtime behavior,
- known scope boundaries of the prototype.

6.2. Test Cases

Table 3: Final prototype test cases

ID	Test description	Expected result
TC-01	Local thermal frame or stream ingestion	Frames are accepted from a local source and forwarded to the pipeline without external service dependence.

ID	description	result
TC-02	Multi-object thermal detection	The detector identifies relevant targets such as people or vehicles and returns bounding boxes with coarse labels.
TC-03	Region-of-interest extraction	Each detection is cropped correctly and isolated for second-stage reasoning.
TC-04	Vehicle semantic analysis	The reasoner returns a richer interpretation than the plain label, including likely category and heat-related status clues.
TC-05	Human or armed-target semantic analysis	The reasoner produces a textual interpretation of posture or carried-object clues when present.
TC-06	Structured JSON generation	Detector outputs and reasoner outputs are merged into machine-readable structured records.
TC-07	Local presentation or logging	Final outputs are available for local review through logs, dashboard-style presentation, or both.
TC-08	Closed-network execution path	Runtime operation does not require external internet connectivity or third-party API calls.
TC-09	Containerized runtime startup	The packaged environment initializes with the required local dependencies and runtime services for prototype execution.
TC-10	Local model initialization	Detector and reasoning components load locally without remote service fallback before frame processing begins.
TC-11	No-detection handling and conditional execution	Frames with no relevant detection do not crash the pipeline and do not force unnecessary second-stage reasoning.

The deployment-related cases above They verify that the system can be started, initialized, and operated locally as an edge-oriented workflow.

6.3. Observed Bugs and Weaknesses

The most significant weakness observed in the reconstructed validation record was the inconsistency of very fine-grained recognition from thermal crops. This is not surprising: the project deliberately operates in a domain with weak texture and limited identifying detail.

6.4. Potential Enhancements

The test results suggest clear future improvements:

- expand thermal-domain fine-tuning data for better object-specific reasoning,
- constrain prompts and output schemas to reduce over-general interpretation,
- preserve automated test scripts and benchmark logs as first-class project artifacts,
- formalize Jetson-oriented profiling for FPS, RAM usage, and startup stability,
- perform longer-duration embedded and field-style evaluation on the final target hardware.

7. Engineering Impact, Ethics, and Contemporary Issues

The project is not only a technical exercise in computer vision. It also has broader global, economic, environmental, societal, and ethical implications. Because the application domain touches surveillance and security, these considerations are part of the design itself rather than an afterthought.

7.1. Global and Strategic Impact

At a global level, the project addresses a widely relevant engineering problem: how to obtain useful situational awareness from difficult imagery without depending on remote cloud infrastructure. This matters not only in defense settings, but also in border monitoring, critical-infrastructure protection, disaster assessment, and emergency response in degraded visual environments.

The architectural emphasis on local processing and structured interpretation also contributes to the broader trend toward edge intelligence. Instead of sending raw mission imagery to a remote platform, the system tries to extract meaning close to the data source.

7.2. Economic Impact

The project was intentionally designed around open-source models and frameworks. This has clear economic consequences:

- lower licensing cost compared with proprietary end-to-end platforms,
- lower recurring bandwidth and cloud-inference cost due to local processing,
- easier technology transfer to future academic or industrial teams because the toolchain is based on widely available frameworks.

At the same time, the project also demonstrates that “open-source” does not mean “free of engineering cost.” Thermal adaptation, fine-tuning, validation, and deployment all still require time, hardware, and expert effort.

7.3. Environmental Impact

The environmental profile of the system is mixed. Local edge inference can reduce the energy and infrastructure cost associated with constant remote data transfer and cloud processing. However, multimodal reasoning models are computationally expensive, and their training or adaptation can still consume substantial compute resources.

This is why optimization techniques such as parameter-efficient fine-tuning and quantization are important. They are not only performance strategies, but also part of a more responsible use of computing resources.

7.4. Societal and Ethical Impact

The potential societal benefit of the project is improved operator awareness. A system that helps distinguish between "an object exists" and "this object is likely relevant or risky" can reduce fatigue and help prioritize attention in time-sensitive environments.

At the same time, surveillance technology creates ethical risk. A system that interprets people, vehicles, and possible threats must not be treated as an unquestionable authority. The project therefore follows a human-in-the-loop philosophy consistent with professional engineering ethics [19, 20]:

- the system informs human decision-making rather than replacing it,
- outputs are treated as analytical suggestions, not final truth,
- privacy, security, and data sovereignty are design requirements, not optional features.

7.5. Contemporary Issues Related to the Project Topic

Several contemporary issues are directly relevant to the project.

- **AI in security and defense:** there is growing pressure to increase automation, but this increases the importance of accountability and operator oversight.
- **Bias in multimodal models:** general VLMs are shaped by RGB-heavy training corpora, which can lead to systematic misunderstanding of thermal scenes.
- **Data sovereignty:** many current AI systems assume cloud-based deployment, which is unacceptable in some operational environments.
- **Open-source model governance:** open models improve accessibility and reproducibility, but they also require careful evaluation before domain-sensitive use.
- **Explainability vs. fluency:** a model can produce convincing language that is not always well grounded in thermal evidence, so output structure and operator review remain essential.

7.6. Professional Responsibility

From an engineering ethics perspective, the project reinforces three responsibilities:

- build systems that reduce harm rather than simply increasing technical capability,

- preserve transparency about what the prototype can and cannot do,
- treat validation, documentation, and human oversight as core deliverables.

For this reason, the final report does not present the system as an autonomous authority. It is more accurate and more responsible to describe it as a secure, locally deployable semantic assistant for thermal imagery.

8. Final Project Status, Limitations, and Future Work

The project concluded with a working implementation at prototype level. The core hybrid pipeline was completed conceptually and functionally: detection, region isolation, reasoning, structured output, and closed-network design were all part of the final system. What remains unfinished is not the basic architecture, but the broader packaging and qualification work required for a fully released or field-hardened product.

8.1. Final Status Overview

Table 4: Final project status by work package

Work package	Status	Notes
Problem analysis and literature review	Complete	Background study established the need for a detector-plus-reasoner architecture.
Requirements and constraints definition	Complete	Functional, non-functional, and ethical constraints were documented early and preserved in the final scope.
Two-stage architecture design	Complete	The final system architecture was fixed as "Gözcü + Analist."
Working prototype implementation	Complete	Core detection, cropping, reasoning, and structured output workflow reached prototype form.
Closed-network deployment concept	Complete	The runtime design avoids external cloud dependence and supports local execution.
Archived automated benchmark evidence	Pending	Final report relies on retained system-level validation rather than a preserved benchmark bundle.

8.2. Main Limitations

The final prototype has several important limitations.

- **Fine-grained thermal ambiguity:** exact make/model recognition and highly specific semantic claims remain unreliable when the crop itself contains limited detail.
- **Dataset dependence:** output quality depends heavily on the representativeness of the thermal training and adaptation data.
- **Edge-compute pressure:** a strong VLM is useful but computationally expensive, especially when low latency is required.

- **Precision-performance trade-off:** FP16, quantized, or engine-compiled execution paths can improve throughput, but they must be checked carefully against any loss of semantic fidelity or detector stability.
- **Packaging complexity:** cross-architecture builds, container runtime mapping, and target-specific dependency management increase engineering effort even when the high-level model logic is already working.

These limitations do not invalidate the project, but they define its maturity level accurately.

8.3. Low-Level Engineering Constraints

The low-level deployment view introduces several practical constraints that are less visible in high-level architecture diagrams.

- Increasing semantic depth usually increases latency and RAM pressure.
- More aggressive visual enrichment strategies, such as multiple crops or repeated target passes, may improve interpretive quality but are expensive on smaller embedded devices.
- Detector acceleration and reasoning acceleration do not scale identically; a pipeline may have a fast first stage and still bottleneck on the second stage.
- Visualization and local container execution can require explicit host-device integration, which is operationally simple in principle but non-trivial to standardize robustly.

These are implementation realities rather than conceptual flaws, but they directly affect what counts as a successful edge deployment.

8.4. Future Work

The most useful future extensions are:

1. Build a richer thermal instruction-tuning dataset for the reasoning stage, especially around vehicle subtype, weapon clue, and posture interpretation.
2. Add stricter structured prompting and schema validation to reduce overly broad or weakly grounded textual interpretations.
3. Perform repeatable latency, memory, and stability measurements on the final target edge hardware under longer-running workloads.
4. Publish a reproducible containerized deployment workflow, including Jetson-targeted runtime configuration, startup instructions, and optimization settings.
5. Extend the output layer toward mission logging, audit trails, and more formal user-interface integration.

8.5. Overall Final Assessment

Despite the limitations above, the final assessment of the project is positive. The project successfully demonstrated that a hybrid architecture is a practical way to move from thermal

detection toward thermal understanding in a closed-network environment. That architectural result is the main contribution of the work.

9. Project Availability and Submission Package

9.1. Repository, Public Web Links, and Distribution Materials

EKLENECEK.

Table 5: Project links

Item	Entry
GitHub Repository	`EKLENECEK`
Web Page	`EKLENECEK`

9.2. DVD-ROM and Manual Contents

EKLENECEK.

- source code or the final approved software snapshot,
- model configuration files and dependency instructions,
- sample input data or demonstration frames where allowed,
- output examples in JSON and screenshot form,
- a user manual covering installation, execution, configuration, and known limitations.

References

[1] G. Jocher et al., *Ultralytics YOLOv8*, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[2] J. Bai et al., "Qwen2-VL: A Versatile and Strong Vision-Language Model," *arXiv preprint arXiv:2407.13521*, 2024.

[3] H. Liu, C. Li, Y. Li, B. Li, Y. Zhang, S. Shen, and Y. J. Lee, "LLaVA-NeXT: Improved reasoning, OCR, and world knowledge," 2024. [Online]. Available: https://llava-vl.github.io/blog/2024-01-30-llava-next/

[4] Z. Chen et al., "InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Linguistic Tasks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 24185–24198, 2024.

[5] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint arXiv:2106.09685*, 2021.

[6] Docker Inc., *Docker Buildx Documentation*, 2024. [Online]. Available: https://docs.docker.com/build/buildx/

[7] NVIDIA Corporation, *NVIDIA TensorRT Documentation*, 2024. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/

[8] NVIDIA and PyTorch Contributors, *Torch-TensorRT Documentation*, 2024. [Online]. Available: https://pytorch.org/TensorRT/

[9] Teledyne FLIR, *FLIR ADAS Dataset*, 2018. [Online]. Available: https://www.flir.com/oem/adas/adas-dataset-form/

[10] S. Hwang, J. Park, N. Kim, Y. Choi, and I. S. Kweon, "Multispectral Pedestrian Detection: Benchmark Dataset and Baseline," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015.

[11] D. Du et al., "HIT-UAV: A High-Altitude Infrared Thermal Dataset for Unmanned Aerial Vehicle-Based Object Detection," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–13, 2022.

[12] Y. Sun, B. Cao, P. Zhu, and Q. Hu, "Drone-based RGB-Infrared Cross-Modality Vehicle Detection via Uncertainty-Aware Learning," *IEEE Transactions on Circuits and Systems for Video Technology*, 2022. [Online]. Available: https://github.com/VisDrone/DroneVehicle

[13] X. Jia, C. Zhu, M. Li, W. Tang, and W. Zhou, "LLVIP: A Visible-Infrared Paired Dataset for Low-Light Vision," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, pp. 3496–3504, 2021.

[14] H. V. Pham et al., "Benchmarking Jetson Edge Devices with an End-to-End Video-based Anomaly Detection System," 2023.

[15] F. Munir, S. Azam, and M. Jeon, "SSTN: Self-Supervised Domain Adaptation Thermal Object Detection for Autonomous Driving," IEEE, 2021.

[16] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[17] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[18] NVIDIA Corporation, *NVIDIA Jetson Platform*, 2024. [Online]. Available: https://developer.nvidia.com/embedded-computing

[19] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*, 2018. [Online]. Available: https://www.acm.org/code-of-ethics

[20] IEEE, *IEEE Code of Ethics*, 2020. [Online]. Available: https://www.ieee.org/about/corporate/governance/p7-8.html